SVD-LLM: Technical Report

Truncation-aware Singular Value Decomposition
for Large Language Model Compression

Independent Reproduction of *Wang et al. (ICLR 2025)*
arXiv:2403.07378

Target model: LLaMA-7B

Compression ratios: 20%, 40%, 60%, 80%

Evaluation: WikiText-2 / C4 perplexity + 8 downstream tasks

Hardware	6× NVIDIA A800 80 GB PCIe
Software	PyTorch 2.6.0, Transformers 4.57.1, lm-eval 0.4.9, PEFT 0.6.0
Date	March 13, 2026

Abstract

This report provides a self-contained description and independent reproduction of **SVD-LLM** (Wang et al., ICLR 2025), a post-training low-rank compression method for large language models. SVD-LLM introduces two key techniques: (1) *truncation-aware data whitening*, which transforms the input activations so that truncating the smallest singular values of the transformed weight matrix is provably optimal in the output-error sense; and (2) *sequential low-rank approximation*, a two-stage LoRA fine-tuning procedure that alternately updates the two factor matrices to recover accuracy lost during truncation. We reproduce the method from the paper alone—without referencing the official codebase—and evaluate on LLaMA-7B at compression ratios of 20%, 40%, 60%, and 80%. The SVD-LLM(W) (whitening-only) variant matches or outperforms the paper's reported perplexity at every ratio; downstream accuracy on six classification benchmarks agrees within ± 0.05. We provide complete mathematical derivations, pseudocode, implementation details, and a full notation reference.

Contents

1 Notation **3**

2 Introduction and Motivation **3**

3 Problem Formulation **4**
3.1 Compression Objective . . . 4
3.2 Compression Ratio and Rank . . . 4
3.3 Target Linear Layers . . . 4

4 Vanilla SVD and Its Failure Mode **5**
4.1 Standard Truncated SVD . . . 5
4.2 Why Weight-Optimal $\neq$ Output-Optimal . . . 5

5 SVD-LLM Core Method **5**
5.1 Truncation-Aware Data Whitening . . . 5
5.2 Sequential Low-Rank Approximation . . . 7

6 CompressedLinear Architecture **8**

7 Complete Algorithms **9**

8 Implementation Details **9**
8.1 Streaming Covariance Collection . . . 9
8.2 Single-Pass vs. Layer-wise Collection . . . 10
8.3 Early-Exit Hook . . . 10
8.4 Temporary File Spilling . . . 10
8.5 Tokenizer Compatibility . . . 10

9 Experiments and Reproduction Results **10**
9.1 Experimental Setup . . . 10
9.2 Perplexity Results . . . 11
9.3 Downstream Task Results . . . 12
9.4 Impact of Sequential LoRA . . . 13

10 Discussion and Limitations **13**

11 Code–Math Symbol Correspondence **14**

References **14**

1 Notation

Table 1 summarises the symbols used throughout this report.

Table 1: Notation reference.

Symbol	Meaning
Dimensions & indices	
d	Output dimension of a linear layer
n	Input dimension of a linear layer
r	Target rank after truncation
r_{lora}	LoRA adapter rank (default 32)
L	Number of Transformer layers (32 for LLaMA-7B)
N	Number of calibration token activations ($256 \times 2048 = 524{,}288$)
Matrices & vectors	
$W \in \mathbb{R}^{d \times n}$	Original weight matrix of a linear layer
$\hat{W}$	Rank-r approximation of W
$x \in \mathbb{R}^n$	Input activation vector to a linear layer
$X \in \mathbb{R}^{N \times n}$	Matrix of stacked activation row vectors
$C \in \mathbb{R}^{n \times n}$	Activation covariance $\frac{1}{N} X^T X$
$L_{\text{chol}} \in \mathbb{R}^{n \times n}$	Lower-triangular Cholesky factor of C ($C = L_{\text{chol}} L_{\text{chol}}^T$)
$S \in \mathbb{R}^{n \times n}$	Upper-triangular factor $S = L_{\text{chol}}^T$ (code convention)
$z = L_{\text{chol}}^{-1} x$	Whitened activation (satisfies $\mathbb{E}[zz^T] = I$)
SVD components	
$U \in \mathbb{R}^{d \times d}$	Left singular vectors of $W L_{\text{chol}}$
Σ	Diagonal matrix of singular values $\sigma_1 \geq \sigma_2 \geq \cdots$
$V \in \mathbb{R}^{n \times n}$	Right singular vectors of $W L_{\text{chol}}$
U_r, Σ_r, V_r	Top-r truncations
Compressed factors	
$W'_u = U_r \Sigma_r^{1/2} \in \mathbb{R}^{d \times r}$	Left factor (stored in `CompressedLinear.second`)
$W'_v = \Sigma_r^{1/2} V_r^T L_{\text{chol}}^{-1} \in \mathbb{R}^{r \times n}$	Right factor (stored in `CompressedLinear.first`)
Metrics & parameters	
$R_w \in [0, 1]$	Compression ratio (fraction of parameters removed)
$\mathcal{L}_i$	Output error from truncating the i-th singular value
$\lVert \cdot \rVert_F$	Frobenius norm
ϵ	Cholesky regularisation constant (10^{-6})

2 Introduction and Motivation

Large language models (LLMs) such as LLaMA-7B contain billions of parameters, making inference deployment challenging in terms of memory, latency, and energy consumption. **Post-training compression** aims to reduce model size without retraining from scratch. Three major families exist:

Table 2: Post-training compression methods for LLMs.

Family	Representatives	Key Idea
Quantization	GPTQ [5], AWQ [6]	Reduce bit-width (FP16→INT4)
Pruning	SparseGPT [7], Wanda [8]	Remove weights/heads
Low-rank	**SVD-LLM** [1], ASVD [9], FWSVD [10]	Decompose W into AB

SVD-LLM belongs to the low-rank family and addresses a fundamental flaw shared by prior SVD-based methods: *truncating the smallest singular values of W does not minimize the output reconstruction error when the input activations are non-isotropic.* SVD-LLM fixes this via two contributions:

1. **Truncation-Aware Data Whitening (§5.1):** Uses calibration data to construct a whitening transform so that truncating the smallest singular value becomes *exactly equivalent* to minimizing output error.
2. **Sequential Low-Rank Approximation (§5.2):** After compression, alternately fine-tunes the two low-rank factor matrices with LoRA on the Alpaca dataset, recovering accuracy lost due to truncation.

3 Problem Formulation

3.1 Compression Objective

Each linear layer in a Transformer computes $y = Wx + b$ with $W \in \mathbb{R}^{d\times n}$. Low-rank compression seeks a rank-r approximation $\hat{W}$ minimizing the *output* reconstruction error under the calibration distribution $\mathcal{D}$:

$$\min_{\hat{W},\, \mathrm{rank}(\hat{W})\le r} \mathbb{E}_{x\sim\mathcal{D}}\left[\left\|(W-\hat{W})\,x\right\|_2^2\right]. \tag{1}$$

Key distinction

The objective is the output-space ℓ_2 error, **not** the weight-space Frobenius norm $\|W-\hat{W}\|_F^2$. This distinction is the methodological cornerstone of SVD-LLM.

3.2 Compression Ratio and Rank

The original layer stores $d \cdot n$ parameters. The rank-r factorisation $\hat{W} = AB$ ($A \in \mathbb{R}^{d\times r}$, $B \in \mathbb{R}^{r\times n}$) stores $(d+n)r$. The compression ratio R_w (fraction of parameters removed) is

$$R_w = 1 - \frac{(d+n)\,r}{d\,n}, \qquad r = \max\left(1,\ \left\lfloor (1-R_w)\,\frac{d\,n}{d+n}\right\rfloor\right). \tag{2}$$

Table 3: Target ranks for LLaMA-7B at four compression ratios.

R_w	$W \in \mathbb{R}^{4096\times4096}$	$W \in \mathbb{R}^{4096\times11008}$
20%	1638	2388
40%	1228	1791
60%	819	1194
80%	409	597

3.3 Target Linear Layers

Each of the 32 Transformer layers in LLaMA-7B contains 7 linear sub-layers (Table 4). SVD-LLM compresses all $32 \times 7 = 224$ matrices independently; `lm_head` and embedding layers are left untouched.

Table 4: Linear sub-layers compressed per Transformer block.

Module	Sub-layer	Weight shape	Function
Self-attention	`q_proj`	4096×4096	Query projection
	`k_proj`	4096×4096	Key projection
	`v_proj`	4096×4096	Value projection
	`o_proj`	4096×4096	Output projection
MLP (SwiGLU)	`gate_proj`	11008×4096	Gating projection
	`up_proj`	11008×4096	Up projection
	`down_proj`	4096×11008	Down projection

4 Vanilla SVD and Its Failure Mode

4.1 Standard Truncated SVD

The SVD of W is $W = U\Sigma V^T = \sum_i \sigma_i u_i v_i^T$. The rank-$r$ truncation $\hat{W}_{\text{SVD}} = U_r \Sigma_r V_r^T$ is optimal in the Frobenius sense (Eckart–Young–Mirsky theorem [2]):

$$\hat{W}_{\text{SVD}} = \arg\min_{\text{rank}(\hat{W}) \le r} \left\| W - \hat{W} \right\|_F^2 = \arg\min_{\text{rank}(\hat{W}) \le r} \sum_{i>r} \sigma_i^2. \tag{3}$$

4.2 Why Weight-Optimal $\neq$ Output-Optimal

The output error from truncating the i-th singular triplet is

$$\mathcal{L}_i = \mathbb{E}\left[\left\| \sigma_i u_i (v_i^T x) \right\|_2^2 \right] = \sigma_i^2 \cdot v_i^T \underbrace{\mathbb{E}[xx^T]}_{=C} v_i = \sigma_i^2 \cdot v_i^T C\, v_i, \tag{4}$$

where $C \in \mathbb{R}^{n \times n}$ is the activation covariance. Truncating the smallest σ_i minimizes $\mathcal{L}_i$ *only if* $v_i^T C v_i$ is constant across i—i.e., $C \propto I$.

The problem with LLMs

In practice, LLM activations are highly anisotropic: the per-dimension variance of x can vary by a factor of 10^2 or more. Hence $v_i^T C v_i$ is far from constant, and truncating the smallest σ_i can incur a *large* output error on high-variance directions. Table 8 confirms this empirically: vanilla SVD yields PPL $> 20{,}000$ at 20% compression, whereas SVD-LLM achieves 7.84.

5 SVD-LLM Core Method

5.1 Truncation-Aware Data Whitening

Goal. Construct a linear reparameterisation of the input such that the activation covariance becomes I, making truncation of the smallest singular value *strictly equivalent* to minimizing the output error.

Step 1 — Activation Covariance

Using N calibration tokens drawn from WikiText-2 (256 sequences of length 2048, giving $N = 524{,}288$ token activations per layer), compute:

$$C = \frac{1}{N} \sum_{k=1}^{N} x_k x_k^T = \frac{1}{N} X^T X \in \mathbb{R}^{n \times n}. \tag{5}$$

Implementation. Storing X requires $\sim$22 GB for one 11008-dimensional layer, which is infeasible. Instead, a forward hook streams and accumulates $X^T X$ on-GPU in FP32:

Listing 1: Streaming covariance accumulation.

```
def hook_fn(module, input, output):
    x = input[0].detach().float()          # (B, T, n)
    x = x.reshape(-1, x.shape[-1])         # (B*T, n)
    accumulators[key]["XtX"] += x.T @ x    # (n,n) on GPU FP32
    accumulators[key]["N"]   += x.shape[0]
```

GPU memory: $6 \times 32 \times 4096^2 \times 4\,\text{B} \approx 12\,\text{GB}$ (attention) $+32 \times 11008^2 \times 4\,\text{B} \approx 15\,\text{GB}$ (MLP) $=\sim$27 GB total.

Step 2 — Cholesky Whitening Factor

Perform Cholesky decomposition $C = L_{\text{chol}} L_{\text{chol}}^T$ (the code stores $S = L_{\text{chol}}^T$, so $C = S^T S$). The whitening transform is $z = L_{\text{chol}}^{-1} x$:

$$\mathbb{E}[zz^T] = L_{\text{chol}}^{-1}\, \mathbb{E}[xx^T]\, L_{\text{chol}}^{-T} = L_{\text{chol}}^{-1}\, L_{\text{chol}}\, L_{\text{chol}}^T\, L_{\text{chol}}^{-T} = I. \tag{6}$$

Numerical stability. Two-level regularisation:

1. Always add ϵI ($\epsilon = 10^{-6}$) before factoring.
2. If Cholesky still fails (rank-deficient C), add $\frac{\text{tr}(C)}{n} \cdot I$ and retry.

Step 3 — SVD in the Whitened Space

Since $x = L_{\text{chol}}\, z$, the output is $y = Wx = (W\, L_{\text{chol}})\, z$. Apply SVD to $W\, L_{\text{chol}}$ (not W):

$$W\, L_{\text{chol}} = U \Sigma V^T. \tag{7}$$

The output error from truncating the i-th component is now

$$\mathcal{L}_i = \sigma_i^2 \cdot v_i^T \underbrace{\mathbb{E}[zz^T]}_{=I} v_i = \sigma_i^2 \cdot \underbrace{\|v_i\|^2}_{=1} = \sigma_i^2. \tag{8}$$

Main result

Proposition 1. *After whitening, the output error from truncating singular values $r+1$ through $\min(d, n)$ is exactly*

$$\mathbb{E}\left[\left\|(W - \hat{W})x\right\|_2^2\right] = \left\|(W - \hat{W})L_{\text{chol}}\right\|_F^2 = \sum_{i>r} \sigma_i^2, \tag{9}$$

where σ_i are the singular values of $W\, L_{\text{chol}}$. Retaining the top-r singular values is therefore optimal.

Step 4 — Symmetric Singular-Value Split

Retaining the top-r components and mapping back:

$$W\, L_{\text{chol}} \approx U_r \Sigma_r V_r^T \quad \Longrightarrow \quad W \approx U_r \Sigma_r V_r^T L_{\text{chol}}^{-1}. \tag{10}$$

SVD-LLM distributes Σ_r *symmetrically*:

$$\boxed{W_u' = U_r\, \Sigma_r^{1/2} \in \mathbb{R}^{d \times r}, \qquad W_v' = \Sigma_r^{1/2}\, V_r^T\, L_{\text{chol}}^{-1} \in \mathbb{R}^{r \times n}.} \tag{11}$$

Verification: $W'_u W'_v = U_r \Sigma_r^{1/2} \cdot \Sigma_r^{1/2} V_r^T L_{\text{chol}}^{-1} = U_r \Sigma_r V_r^T L_{\text{chol}}^{-1} \approx W$. The symmetric split ensures both factors have similar numerical scales, producing more balanced gradients during the subsequent LoRA fine-tuning.

Listing 2: Core whitening SVD implementation.

```
S, S_inv = compute_whitening_matrix_from_covariance(C)
WS = W @ S.T                    # W @ L_chol  (S=L^T, so S.T=L)

U, Sigma, Vh = torch.linalg.svd(WS, full_matrices=False)
U_r, Sigma_r, Vh_r = U[:, :r], Sigma[:r], Vh[:r, :]

sqrt_sigma = torch.sqrt(Sigma_r)
A = U_r * sqrt_sigma.unsqueeze(0)                # W'_u: (d,r)
B = sqrt_sigma.unsqueeze(1) * (Vh_r @ S_inv.T)   # W'_v: (r,n)
```

Complete Derivation Chain

Objective:	$\min_{\hat{W}} \mathbb{E}\left[\left\|(W-\hat{W})x\right\|_2^2\right]$
Covariance:	$C = \mathbb{E}[xx^T] = \frac{1}{N} X^T X$
Cholesky:	$C = L_{\text{chol}} L_{\text{chol}}^T$
Whitening:	$z = L_{\text{chol}}^{-1} x, \quad \mathbb{E}[zz^T] = I$
Substitution:	$\left\|(W-\hat{W})x\right\|^2 = \left\|(W-\hat{W})L_{\text{chol}}\, z\right\|^2$
SVD:	$W\, L_{\text{chol}} = U \Sigma V^T$
Error:	$\mathbb{E}\left[\left\|(W-\hat{W})x\right\|^2\right] = \sum_{i>r} \sigma_i^2$
Truncation:	$W\, L_{\text{chol}} \approx U_r \Sigma_r V_r^T$
Back-projection:	$W \approx U_r \Sigma_r V_r^T L_{\text{chol}}^{-1}$
Symmetric split:	$W'_u = U_r \Sigma_r^{1/2}, \quad W'_v = \Sigma_r^{1/2} V_r^T L_{\text{chol}}^{-1}$

5.2 Sequential Low-Rank Approximation

Although whitening SVD minimizes information loss during truncation, at high compression ratios (60–80%) perplexity still degrades dramatically (see Figure 1). The second contribution of SVD-LLM is to recover accuracy via a two-stage LoRA fine-tuning procedure.

Why Not Joint Optimisation?

Simultaneously optimizing W'_u and W'_v is a *bilinear* problem:

$$\min_{W'_u, W'_v} \mathcal{L}(W'_u W'_v). \tag{12}$$

Due to multiplicative coupling this is non-convex. Fixing one factor makes the problem convex (linear) in the other. SVD-LLM exploits this via *alternating* optimisation with LoRA adapters.

Alternating (Sequential) Strategy

Stage 1 — LoRA$_u$**:** Freeze all W'_v (`.first`). Attach LoRA(r_{lora}=32) to all W'_u (`.second`) and fine-tune for 1 epoch on Alpaca-cleaned [4]. Merge LoRA weights: $W'_u \leftarrow W'_u + B_{\text{lora}} A_{\text{lora}}$.

Stage 2 — LoRA$_v$**:** Freeze updated W'_u. Attach LoRA to all W'_v and fine-tune for 1 epoch. Merge LoRA weights: $W'_v \leftarrow W'_v + B'_{\text{lora}} A'_{\text{lora}}$.

Hyperparameters

Table 5: Sequential LoRA fine-tuning configuration.

Parameter	**Value**
Fine-tuning data	Alpaca-cleaned 50K (`yahma/alpaca-cleaned`) [4]
LoRA rank r_{lora}	32
LoRA α	64 (effective scaling $\alpha/r = 2$)
LoRA dropout	0.05
Learning rate	2×10^{-4} (cosine, warmup ratio 0.03)
Epochs per stage	1
Effective batch size	4×4 (grad accum) $= 16$
Max sequence length	256
Precision	FP16 mixed
Gradient checkpointing	Enabled

Alpaca Data Processing

Loss is computed only on the response tokens; prompt tokens are masked with -100 (ignored by `CrossEntropyLoss`). Samples where the entire response is truncated are filtered out before training.

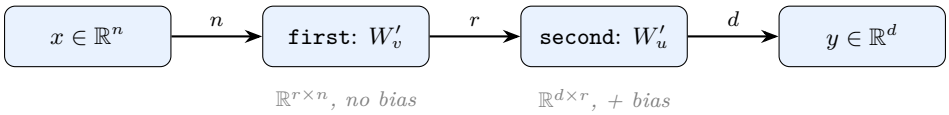
SVD-LLM(W) vs. SVD-LLM

Table 6: Difference between the two variants.

	SVD-LLM(W)	**SVD-LLM**
Stage A (compression)	Whitening + SVD truncation	Same
Stage B (update)	**None**	Sequential LoRA
Calibration data	WikiText-2 $\times$ 256	Same
Fine-tuning data	—	Alpaca 50K
Compression time	$\sim$10 min	$\sim$10 min + $\sim$1 h

6 CompressedLinear Architecture

The original $y = Wx + b$ is replaced by two successive smaller layers:



Parameter count:

$$\underbrace{d \cdot n + d}_{\text{original}} \longrightarrow \underbrace{r \cdot n + r \cdot d + d}_{\text{compressed}} = (n + d)\, r + d. \tag{13}$$

Before saving, all `CompressedLinear` modules are merged back to standard `nn.Linear` via $W_{\text{merged}} = W'_u W'_v$, keeping full compatibility with the HuggingFace serialisation API.

7 Complete Algorithms

Algorithm 1 SVD-LLM(W) — Whitening Compression Only

Input: Model M, calibration data $\mathcal{D}_{\text{calib}}$ (256 sequences, seqlen = 2048), compression ratio R_w
Output: Compressed model M'

for each linear sub-layer $W \in \mathbb{R}^{d \times n}$ **do**
  $r \leftarrow \max(1, \lfloor (1 - R_w)\, dn/(d+n) \rfloor)$
end for
Register forward hooks on all $32 \times 7 = 224$ sub-layers
for each mini-batch $B_t \in \text{batch}(\mathcal{D}_{\text{calib}}, 4)$ **do** ▷ 64 passes
  $M(B_t)$ *hooks accumulate $X^T X$ and N on GPU*
end for
Remove all hooks
for $l = 0, 1, \ldots, 31$ **do**
  for each sub-layer $\in$ {q, k, v, o, gate, up, down} **do**
    $C \leftarrow X^T X / N + \epsilon I$
    $L_{\text{chol}} \leftarrow \text{cholesky}(C)$
    $U, \Sigma, V^T \leftarrow \text{SVD}(W \cdot L_{\text{chol}})$; truncate to rank r
    $W'_u \leftarrow U_r\, \Sigma_r^{1/2}$; $W'_v \leftarrow \Sigma_r^{1/2}\, V_r^T\, L_{\text{chol}}^{-1}$
    Spill (W'_u, W'_v) to temp disk
  end for
end for
Load all pairs; replace each sub-layer with `CompressedLinear`
return M'

Algorithm 2 SVD-LLM — Whitening + Sequential LoRA

Input: Original model M, $\mathcal{D}_{\text{calib}}$, R_w, fine-tuning data $\mathcal{D}_{\text{ft}}$ (Alpaca 50K)
Output: Fine-tuned compressed model M'_{final}

$M' \leftarrow$ Algorithm 1$(M, \mathcal{D}_{\text{calib}}, R_w)$ ▷ Stage A
▷ **Stage B-1**: LoRA$_u$
Freeze `.first` (W'_v) in all CompressedLinear
Attach LoRA(r_{lora}=32, α=64) to all `.second` (W'_u)
Train 1 epoch on $\mathcal{D}_{\text{ft}}$ (causal LM, prompt masked)
Merge LoRA $\rightarrow M'_u$
▷ **Stage B-2**: LoRA$_v$
Freeze `.second` (W'_u)
Attach LoRA to all `.first` (W'_v)
Train 1 epoch on $\mathcal{D}_{\text{ft}}$
Merge LoRA $\rightarrow M'_{\text{final}}$
return M'_{final}

8 Implementation Details

8.1 Streaming Covariance Collection

Storing the full activation matrix $X \in \mathbb{R}^{N \times n}$ is infeasible ($\sim$22 GB per MLP layer). By exploiting the identity $X^T X = \sum_t B_t^T B_t$, only the $n \times n$ accumulator resides in GPU memory. The hooks accumulate in FP32 on GPU and transfer to CPU FP64 once at the end. This avoids per-batch GPU$\leftrightarrow$CPU synchronisation and is approximately 7$\times$ faster than a CPU-side FP64 accumulation

strategy.

8.2 Single-Pass vs. Layer-wise Collection

Since SVD-LLM(W) does not modify weights during calibration, all 224 covariance matrices can be collected in a single set of forward passes. This reduces the total passes from $32 \times 64 = 2048$ (layer-wise) to just 64 (single-pass)—a 32× reduction. The trade-off is GPU memory: all 224 accumulators must reside simultaneously (∼27 GB).

8.3 Early-Exit Hook

For the layer-wise collection mode (used in debugging and sequential-update pipelines), a `forward_pre_hook` on layer $l+1$ raises an exception to abort the forward pass after layer l. This saves ∼50% of FLOPs on average across all layers, as the expected layer index is $L/2 = 16$.

8.4 Temporary File Spilling

After computing (W'_u, W'_v) for each sub-layer, the tensors are spilled to a temporary directory and freed from GPU. The final replacement pass loads them back one layer at a time. This prevents 224 pairs of factor matrices from accumulating in RAM simultaneously (which would require ∼13 GB at FP16 even at 20% compression).

8.5 Tokenizer Compatibility

The `jeffwan/llama-7b-hf` checkpoint ships with incorrect special token IDs (bos = eos = unk = 0). This causes CUDA indexing errors during padded batch generation in `lm-eval`. The evaluation code patches these from `model.config` (bos = 1, eos = 2) before creating the evaluation wrapper.

9 Experiments and Reproduction Results

9.1 Experimental Setup

Table 7: Environment and datasets.

Category	**Item**	**Value**
Software	Python	3.12.10
	PyTorch	2.6.0+cu124
	Transformers	4.57.1
	lm-eval	0.4.9
	PEFT	0.6.0
Hardware	GPU	6× NVIDIA A800 80 GB PCIe
Data	Calibration	WikiText-2 train, 256 seqs × 2048 tokens
	Fine-tuning	Alpaca-cleaned 50K, max_length = 256
	PPL eval	WikiText-2 test (full) / C4 validation (1100 docs)

9.2 Perplexity Results

Table 8: WikiText-2 and C4 perplexity for LLaMA-7B (↓). Bold = our reproduction.

R_w	Dataset	SVD	FWSVD	ASVD	SVD-LLM(W) (paper)	**SVD-LLM(W) (repro.)**
0%	Wiki	5.68	5.68	5.68	5.68	**5.67**
0%	C4	7.34	7.34	7.34	7.34	**7.20**
20%	Wiki	20061	1727	11.14	7.94	**7.84**
20%	C4	18800	1511	15.93	15.84	**15.65**
40%	Wiki	52489	18156	1407	13.73	**13.17**
40%	C4	47774	12847	1109	75.42	**50.05**
60%	Wiki	105474	32194	57057	66.62	**58.98**
60%	C4	106976	29292	43036	471.83	**378.80**
80%	Wiki	687291	96872	80425	1349	**660.67**
80%	C4	708243	89243	67927	6224	**2641.62**

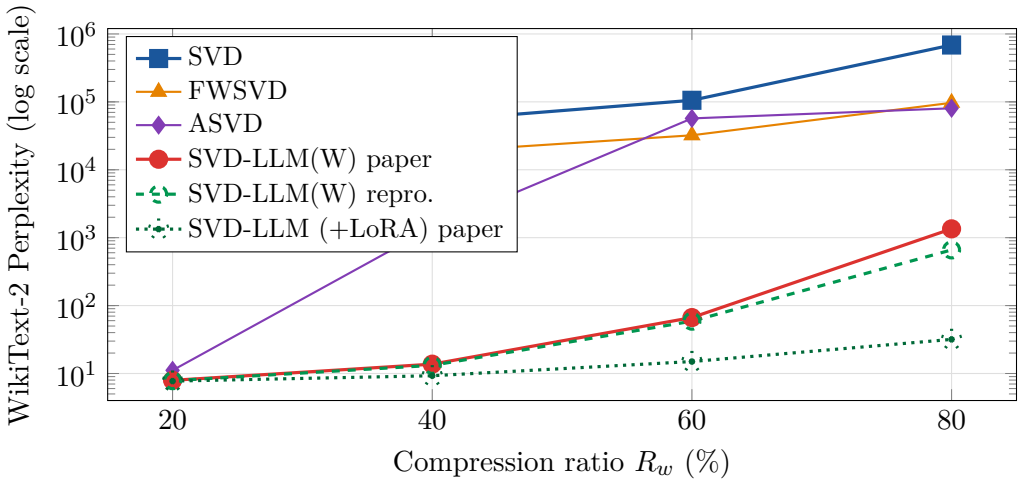


Figure 1: WikiText-2 perplexity vs. compression ratio on LLaMA-7B. SVD-LLM(W) reduces PPL by 3−4 orders of magnitude over vanilla SVD. Sequential LoRA (dotted) further closes the gap at high compression, achieving PPL 31.79 at 80% (vs. 1349 without LoRA).

9.3 Downstream Task Results

Table 9: Downstream accuracy across all compression ratios on LLaMA-7B (↑). SVD-LLM repro. is pending Phase 2 completion.

R_w	Task	SVD-LLM(W) paper	**SVD-LLM(W) repro.**	SVD-LLM paper	**SVD-LLM repro.**
20%	OpenbookQA	0.310	**0.266**	0.330	—
	ARC-Easy	0.620	**0.641**	0.670	—
	WinoGrande	0.610	**0.664**	0.690	—
	HellaSwag	0.450	**0.437**	0.550	—
	PIQA	0.710	**0.690**	0.790	—
	MathQA	0.210	**0.239**	0.260	—
	Avg (6)	0.490	**0.490**	0.550	—
	TruthfulQA†	0.260	**0.390**	0.280	—
	GSM8K	0.050	**0.006**	0.080	—
40%	OpenbookQA	0.250	**0.200**	0.290	—
	ARC-Easy	0.330	**0.457**	0.590	—
	WinoGrande	0.550	**0.575**	0.680	—
	HellaSwag	0.400	**0.330**	0.520	—
	PIQA	0.630	**0.609**	0.690	—
	MathQA	0.120	**0.220**	0.200	—
	Avg (6)	0.380	**0.399**	0.500	—
	TruthfulQA†	0.170	**0.433**	0.240	—
	GSM8K	0.020	**0.000**	0.070	—
60%	OpenbookQA	0.100	**0.132**	0.180	—
	ARC-Easy	0.050	**0.301**	0.420	—
	WinoGrande	0.170	**0.527**	0.440	—
	HellaSwag	0.100	**0.273**	0.310	—
	PIQA	0.210	**0.544**	0.350	—
	MathQA	0.040	**0.218**	0.120	—
	Avg (6)	0.110	**0.333**	0.300	—
	TruthfulQA†	0.010	**0.476**	0.140	—
	GSM8K	0.000	**0.000**	0.040	—
80%	OpenbookQA	0.070	**0.130**	0.110	—
	ARC-Easy	0.030	**0.261**	0.230	—
	WinoGrande	0.040	**0.480**	0.210	—
	HellaSwag	0.020	**0.260**	0.140	—
	PIQA	0.070	**0.523**	0.170	—
	MathQA	0.010	**0.205**	0.080	—
	Avg (6)	0.040	**0.310**	0.160	—
	TruthfulQA†	0.000	**0.501**	0.040	—
	GSM8K	0.000	**0.000**	0.020	—

† TruthfulQA: the original paper reports BLEU scores, whereas our reproduction uses the MC2 metric from `truthfulqa_mc2` in lm-evaluation-harness. The two metrics are not comparable.

Note: Downstream accuracy values may differ from the paper due to lm-evaluation-harness version differences (we use v0.4.9; the paper's version is unspecified). Task implementations, prompting templates, and scoring logic evolve across versions.

9.4 Impact of Sequential LoRA

Table 10: Effect of Sequential LoRA on WikiText-2 PPL (paper numbers).

R_w	SVD-LLM(W)	SVD-LLM	Improvement
20%	7.94	**7.73**	1.03×
40%	13.73	**9.27**	1.48×
60%	66.62	**15.00**	4.44×
80%	1349	**31.79**	42.4×

The benefit of Sequential LoRA grows super-linearly with compression ratio. At 20%, whitening alone already achieves excellent PPL and LoRA provides only marginal improvement (1.03×). At 80%, however, LoRA reduces PPL by 42× (1349 → 31.79), demonstrating that Stage B is critical at aggressive compression levels.

10 Discussion and Limitations

Strengths. SVD-LLM provides a clean, principled framework for low-rank LLM compression. The whitening step is mathematically well-motivated and adds minimal overhead (one extra forward pass for covariance collection + Cholesky + SVD per layer). The sequential LoRA strategy elegantly avoids the non-convexity of joint bilinear optimisation.

Sensitivity to covariance quality. The method's correctness hinges on $\mathbb{E}[zz^T] = I$, which requires an accurate covariance estimate. With only 256 calibration sequences ($N = 524{,}288$ tokens), the sample covariance may be noisy for the 11008×11008 MLP layers (where $N/n \approx 48$). The two-level Cholesky regularisation (§5.1) mitigates rank deficiency but does not eliminate estimation noise. Larger calibration sets or shrinkage estimators (e.g., Ledoit–Wolf) could improve robustness.

Single-model evaluation. This reproduction evaluates only LLaMA-7B. The paper additionally reports results on LLaMA-13B, OPT-6.7B, and Mistral-7B. Generalisation to other architectures (e.g., models with grouped-query attention or mixture-of-experts) remains to be verified.

Comparison with quantisation. Low-rank compression and quantisation are complementary: the former reduces matrix dimensions, the latter reduces per-element precision. At 20% compression (∼10 GB), SVD-LLM(W) achieves comparable PPL to the original but with a larger footprint than 4-bit quantisation (∼3.5 GB). Combining both is a promising direction.

Inference latency. Although this report focuses on compression ratio and accuracy, the `CompressedLinear` architecture (two sequential matmuls instead of one) may not improve wall-clock latency on all hardware. The merged form ($W_{\text{merged}} = W_u' W_v'$) eliminates this overhead but also eliminates the memory saving. Practical deployment requires keeping the factored form and using custom CUDA kernels or compiler optimisation to exploit the reduced arithmetic intensity.

11 Code–Math Symbol Correspondence

Table 11: Mapping between mathematical notation and code.

Math	**Code variable**	**File**
W	`target.weight.data`	`compress_model.py`
$C = X^T X/N$	`XtX / N`	`calibration.py`
L_{chol}	`torch.linalg.cholesky(C)`	`whitening.py:25`
$S = L_{\text{chol}}^T$	`S = L.T`	`whitening.py:31`
WL_{chol}	`WS = W @ S.T`	`whitening.py:83`
U_r, Σ_r, V_r^T	`U_r, Sigma_r, Vh_r`	`whitening.py:86-88`
$\Sigma_r^{1/2}$	`sqrt_sigma`	`whitening.py:91`
W_u'	`A`	`whitening.py:92`
W_v'	`B`	`whitening.py:93`
L_{chol}^{-1}	`S_inv.T`	`whitening.py:93`
r	`compute_rank(d,n,ratio)`	`loader.py:45`
W_v' module	`CompressedLinear.first`	`replace.py:17`
W_u' module	`CompressedLinear.second`	`replace.py:18`

References

[1] X. Wang, Y. Zheng, Z. Wan, M. Zhang. "SVD-LLM: Truncation-aware Singular Value Decomposition for Large Language Model Compression." *ICLR*, 2025. arXiv:2403.07378.

[2] C. Eckart, G. Young. "The approximation of one matrix by another of lower rank." *Psychometrika*, 1(4):211–218, 1936.

[3] E. J. Hu, Y. Shen, P. Wallis, et al. "LoRA: Low-Rank Adaptation of Large Language Models." *ICLR*, 2022.

[4] R. Taori, I. Gulrajani, T. Zhang, et al. "Alpaca: A Strong, Replicable Instruction-Following Model." Stanford CRFM, 2023.

[5] E. Frantar, S. P. Ashkboos, T. Hoefler, D. Alistarh. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers." *ICLR*, 2023.

[6] J. Lin, J. Tang, H. Tang, et al. "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration." *MLSys*, 2024.

[7] E. Frantar, D. Alistarh. "SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot." *ICML*, 2023.

[8] M. Sun, Z. Liu, A. Bair, J. Z. Kolter. "A Simple and Effective Pruning Approach for Large Language Models." *ICLR*, 2024.

[9] Z. Yuan, C. Shang, Y. Wang, et al. "ASVD: Activation-aware Singular Value Decomposition for Compressing Large Language Models." arXiv:2312.05821, 2023.

[10] A. Hsu, W. Hua, Y. Shen, et al. "FWSVD: Fisher-Weighted SVD for Low-Rank LLM Compression." *Findings of ACL*, 2024.